

Overlapping MoE Pre-Barriers with Data Transfer Using In-Network Computing

Abstract

Expert-parallel communication often waits for participating ranks to become ready before sending token data. At small batches, this pre-barrier can occupy a substantial part of Dispatch and Combine. Measurements of DeepEP V2 Direct on two H200 nodes show a pre-barrier duration near 15 μ s at EP8, accounting for about 20% of operator time at eight tokens per rank and about 5% at 128. In Qwen3-30B-A3B decode, pre-barriers account for 15.2–16.7% of the sampled cross-node GPU step.

We present RendezEP, an in-network computing design that overlaps readiness aggregation with data upload. A locally ready rank sends its readiness signal and begins uploading. INC buffers early data and forwards it once the required ranks and destination buffers are ready. Symmetric receive buffers provide known landing positions, while PFC handles staging pressure. We describe the protocol and analyze the resulting execution order. Under simultaneous readiness and unchanged forwarding throughput, early upload can hide one serialized one-way network traversal, less the design’s additional processing cost.

CCS Concepts: • Networks → Network protocol design; In-network processing; • Computer systems organization → Distributed architectures.

Keywords: expert parallelism, mixture-of-experts, in-network computing, synchronization, collective communication, datacenter networks

1 Introduction

Mixture-of-experts (MoE) inference routes token activations to selected experts and combines their outputs. Expert parallelism distributes those experts across GPU ranks, requiring Dispatch and Combine communication at every MoE layer. Communication libraries use GPU-initiated transfers, pre-allocated buffers, and topology-aware paths to move these data efficiently [3, 5].

Small batches make the coordination before transmission particularly important. A pre-barrier may prevent a rank from sending locally available tokens until it has received readiness signals from its peers. The token payload shrinks with batch size, but the readiness exchange still involves the same participating ranks.

Our measurements of DeepEP V2 Direct quantify this cost. With cross-node EP8, the pre-barrier lasts approximately 15 μ s across the measured small-batch configurations. It occupies about 20% of Dispatch and Combine time at eight

tokens per rank, falling to about 5% at 128. In Qwen3-30B-A3B decode, the accumulated pre-barrier intervals account for 15.2–16.7% of the sampled cross-node GPU step, compared with 7.9–9.3% within one node.

The opportunity comes from separating upload from destination delivery. A rank may have data ready to send even while another rank is still preparing its receive buffer. If the network can hold early data, the first rank can begin uploading during this wait. The destination remains protected until its receive buffer is writable.

We present RendezEP, which places readiness aggregation and bounded data staging at an INC node. Ranks announce local readiness and upload without waiting for peer signals to return. INC records the ready participants, buffers early data, and starts forwarding when all required readiness conditions hold. The receiving ranks use preallocated symmetric buffers, so forwarding does not wait for a global receive-count calculation.

This paper contributes a characterization of pre-barrier costs in H200 microbenchmarks and MoE inference, an early-upload protocol with explicit buffer-reuse and backpressure rules, and an operation-level analysis of readiness/data overlap. Following Swift’s separation of processing and network delay [9], the analysis identifies when early upload shortens the full communication operation. The benefit depends on rank readiness times, available staging, and the rate at which INC can forward buffered data.

2 Background and Motivation

2.1 Expert-Parallel Communication

An MoE layer selects up to *topk* experts for each token. Under expert parallelism, Dispatch sends activations to the ranks hosting those experts; Combine returns their outputs to the token owner [3, 5]. Routing determines the destinations and traffic volume. The exchange also carries metadata needed to place tokens into expert input buffers.

Ranks communicate readiness, counts, and routing information through collective operations or point-to-point writes and polling. A readiness signal indicates that the resources needed for the round are available. Communication buffers provide known landing positions; the final arrangement of tokens by expert may depend on exchanged counts.

EP implementations organize these steps differently. NCCL EP HT gathers routing metadata before Dispatch. DeepEP V2 Direct performs a pre-barrier, then processes counts alongside data. NCCL EP LL writes into reserved slots with fine-grained signals [4, 12, 13]. These choices determine what must finish before a rank can send its first token.

2.2 Pre-Barriers in DeepEP V2 Direct

DeepEP V2 Direct uses GPU-initiated transfers and preallocated receive buffers [4]. In Dispatch, separate warps process token counts and layout metadata alongside token transmission. Combine reuses the routing information established during Dispatch.

Both operations perform a pre-barrier before these transfers begin. Each rank waits for the participating ranks to become ready and completes the required local synchronization. Consequently, even a rank with available input data must wait for peer readiness before sending its tokens.

This waiting cost is significant at small batches. In our cross-node EP8 measurements, the pre-barrier remains near 15 μ s, occupying about 20% of operator time at eight tokens per rank and 5% at 128. Section 5 reports the measurements. The opportunity is to begin moving locally ready tokens while readiness is still being collected.

2.3 Opportunities for In-Network Computing

In-network computing lets network devices process data while forwarding it, for example by summing values from several GPUs [15–17]. Such devices can also retain temporary state across packets. For pre-barriers, the useful state is which ranks are ready, together with space to hold data that arrives before forwarding is allowed.

A locally ready rank can upload while INC collects the remaining readiness signals. INC holds early tokens until the required receive buffers are writable, then forwards both buffered and arriving data.

Key insight. A rank can upload locally ready data before all ranks are ready, provided INC buffers the data until the required readiness conditions hold. This overlaps readiness aggregation with the first part of data transfer.

3 Design

RendezEP moves readiness aggregation to INC so that ranks can upload data while waiting for their peers.

3.1 Overview and Assumptions

Figure 1 contrasts the baseline’s wait-before-send sequence with early upload. INC maintains a bitmap of ready ranks and a bounded queue of early data for each communication round. A generation identifies the Dispatch or Combine operation, and a buffer epoch identifies a particular use of a receive buffer. All participating ranks and the traffic controlled by their pre-barrier must be covered by the INC node.

Receive buffers are allocated and registered symmetrically during initialization. INC obtains their layout, address mappings, sizes, and access permissions once. Each token carries routing information and an identifier that locate its slot in the destination’s preallocated buffer. INC can write tokens into these slots before all receive counts are known; the destination can arrange them by expert afterward.

A rank sends READY after its local input is available and the previous use of its receive buffer has finished. READY identifies the rank, generation, and buffer epoch. The node uses it to confirm that the buffer is writable for the current round. Deployment can be within or across nodes; any path that bypasses INC needs the corresponding readiness protection before it writes destination memory.

3.2 Early Upload and Readiness Aggregation

In Direct, a rank waits for peer readiness before sending data. With RendezEP, it sends READY and immediately starts uploading locally available data to INC. This applies to token activations in Dispatch and expert outputs in Combine.

INC records which ranks are ready and buffers data that arrives early. Once all required READY signals have arrived and destination buffers are writable, INC forwards both buffered and incoming data according to their routes.

The benefit comes from moving data while readiness is being collected. While baseline sources are still waiting for peer signals, data can already be in flight to INC or buffered there. Figure 2 illustrates how this overlap can advance data delivery when INC has sufficient buffer space and forwarding bandwidth.

3.3 Buffer Management and Backpressure

A receive buffer remains assigned to its round until the last computation reading it finishes. The next READY confirms that it can be reused. A source buffer remains allocated until sending hardware has finished reading it. Generation and epoch checks keep old messages from authorizing writes into a newer round. INC retires a round’s readiness state after its queued data has drained and all participants have advanced to the next round.

Network-buffer occupancy triggers PFC [8] before overflow, with headroom for data already in flight when a pause is sent. Upload resumes on the same path when capacity becomes available. READY uses a separate traffic class with reserved resources so that DATA pauses cannot block the signals needed to start forwarding. Required control metadata receives the same progress guarantee.

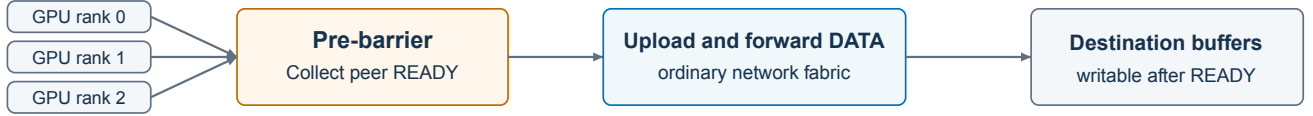
4 Latency Analysis

We model a complete Dispatch or Combine operation as READY, data transfer, and post-barrier. INC starts upload after issuing READY, overlapping data movement with the wait for peer readiness.

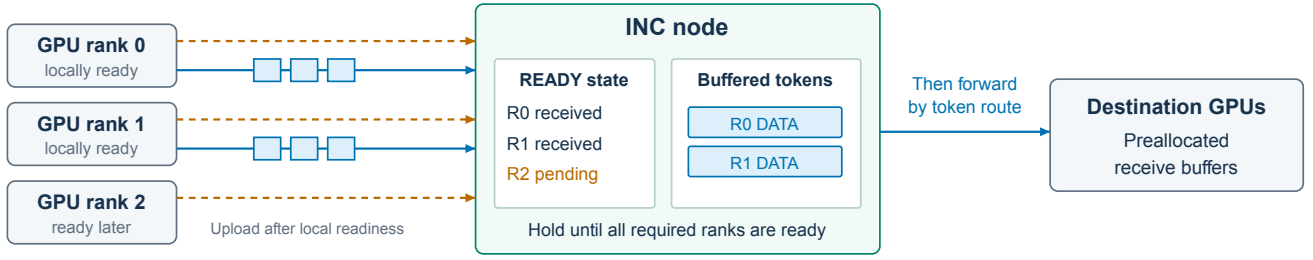
4.1 Three-Stage Model

Let T_{ready} be the pre-barrier duration, T_{data} the data-transfer phase, and T_{post} the post-barrier and subsequent local epilogue. The data phase includes the sender-side work needed to enter the post-barrier. The baseline executes these stages

A Direct: wait, then upload



B INC: upload while collecting READY



Earlier upload; destination writes still wait for readiness.

Figure 1. Early upload overlaps readiness collection. The INC snapshot shows rank 2 still pending and data from ranks 0 and 1 buffered. Forwarding starts after all required ranks are ready and destination buffers are writable.

in sequence:

$$T_{\text{base}} = T_{\text{ready}} + T_{\text{data}} + T_{\text{post}}. \quad (1)$$

Following Swift’s separation of processing and network delay [9], consider a reference with simultaneous rank readiness and comparable communication paths:

$$T_{\text{ready}} = T_{\text{issue}} + L + T_{\text{observe}}. \quad (2)$$

Here T_{issue} is the local work to prepare and issue READY, L is its one-way network traversal, and T_{observe} is the subsequent signal checking and local synchronization before DATA starts. These terms describe the critical execution path; per-rank minimum measurements are reported separately in Section 5.

4.2 Overlapping the Wait for READY

In the baseline, ranks finish the entire READY stage before sending DATA. With INC, each rank issues READY and starts uploading. READY and DATA travel toward the network node together; INC holds any early data until the required signals have arrived and destination buffers are writable. Figure 2 illustrates this change in timing.

Using the same reference data-transfer duration and subsequent completion work, the INC schedule is

$$T_{\text{INC}} = T_{\text{issue}} + T_{\text{data}} + T_{\text{post}} + T_{\text{extra}}. \quad (3)$$

The local issue time remains. T_{extra} includes readiness processing still required at INC or the endpoints and additional delay from buffering or waiting to forward. The ordinary source-to-destination traversal of DATA is already included in T_{data} .

The reference difference is therefore

$$\Delta T = L + T_{\text{observe}} - T_{\text{extra}}. \quad (4)$$

The network component is one serialized READY traversal, L , which is approximately half a network RTT for symmetric paths. The net benefit also depends on how much endpoint observation work is replaced and how much processing and waiting INC adds. Half a network RTT is not half the measured pre-barrier duration.

4.3 Readiness Skew in Practice

In practice, differences in computation and scheduling often make ranks ready at different times. Figure 2(C–D) shows how the overlap works in this case: early ranks upload while later ranks prepare, and INC buffers the data. When the last required READY arrives, INC can start forwarding these tokens instead of waiting for the early ranks to receive peer signals and then begin uploading. The mechanism therefore does not require synchronized arrivals.

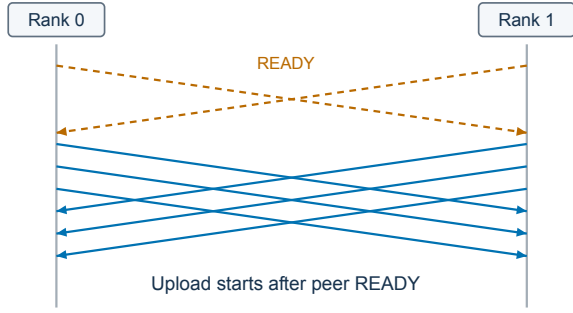
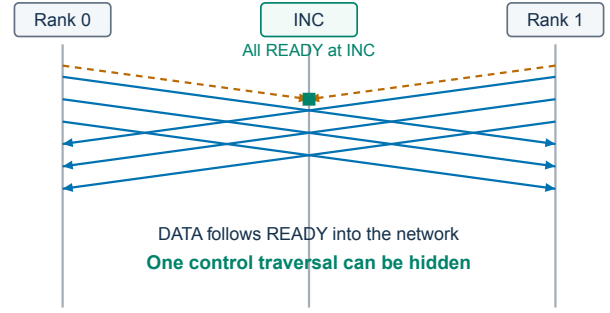
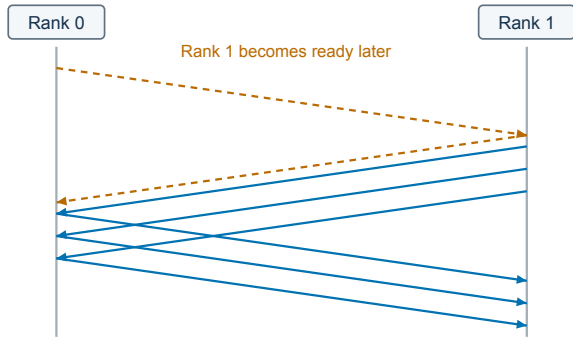
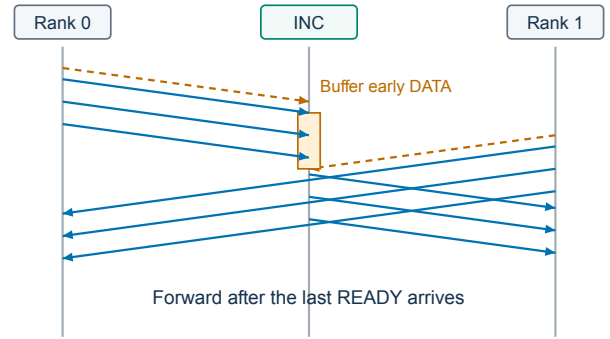
With sufficient staging and forwarding capacity, this upload head start is preserved under readiness skew and shortens the operation when it advances critical-path data delivery. Output queuing and PFC pauses contribute to T_{extra} . The gain depends on the last rank to finish and the available overlap, rather than a fixed fraction of RTT for every arrival pattern.

5 Evaluation

We ask three questions: how pre-barrier cost changes with token count, how it appears across EP configurations, and how much of an MoE serving step it occupies.

5.1 Experimental Setup and Methodology

Each node has eight H200 GPUs connected by NVSwitch and eight 400 GbE RoCE interfaces. Cross-node runs use GIN’s

A Simultaneous readiness: Direct**B Simultaneous readiness: INC****C Unequal readiness: Direct****D Unequal readiness: INC**

Dashed: READY Solid: DATA Schematics show readiness and data delivery; spacing is illustrative.

Figure 2. Readiness and data transmission with simultaneous (top) and unequal (bottom) rank readiness. Dashed arrows are READY; solid arrows are DATA. INC holds early data until the last required READY arrives. The aligned reference can hide one control traversal; actual operation latency also depends on output queuing and subsequent work.

GDAKI backend, traffic class 162, PyTorch 2.13, CUDA 13, NCCL 2.31.2, and DeepEP V2 based on commit 01dc3aaa [4]. Direct is selected with Hybrid disabled; available local communication within Direct is retained.

Microbenchmarks use $H = 7168$, $topk = 8$, 256 experts, BF16, and balanced uniform routing. T denotes tokens per rank. Each configuration has three process runs with 200 measured iterations. For each iteration, we report the shortest pre-barrier across ranks and normalize it by the same rank’s trace-on operator time. Results are averaged over iterations and runs; error bars show variation across run means.

The EP8 token sweep and the EP-size comparison are separate campaigns. Figure 3 uses $T = 8, 32, 64, 128$ from one instrumented image. Table 1 uses its listed points from the original campaign plus a later EP8/T64 batch; its constants are fitted only to those values.

5.2 Pre-Barrier Cost versus Token Count

Figure 3 reports pre-barrier duration and its share within a single Direct Dispatch or Combine operator at EP8. Dispatch and Combine remain near $15 \mu s$ across the measured token counts. Their operator shares fall from 20.57% and 19.84% at

EP	Fitted T	Dispatch	Combine	Max. dev.
4	8/32/96/128	11.65	11.54	0.06
8	8/32/64/96/128	14.96	14.94	0.16
16	8/32/96/128	18.36	18.28	0.09

Table 1. Pre-barrier constants and maximum deviation of per- T run means, in μs . Each T has three process runs. EP8 includes a later $T = 64$ batch with different instrumentation.

$T = 8$ to 4.88% and 5.05% at $T = 128$. Small batches therefore expose a larger fraction of each communication operator to readiness coordination.

5.3 Across EP Configurations

Table 1 summarizes each EP configuration with a constant fitted to all listed token counts. The largest departure of a token-count mean from its configuration’s constant is $0.16 \mu s$, so token count has little effect over these measured ranges.

The fitted duration rises from approximately $11.6 \mu s$ at EP4 to $18.3 \mu s$ at EP16. GPUs per node, destination count, and network concurrency change together across these configurations; the table reports their combined effect.

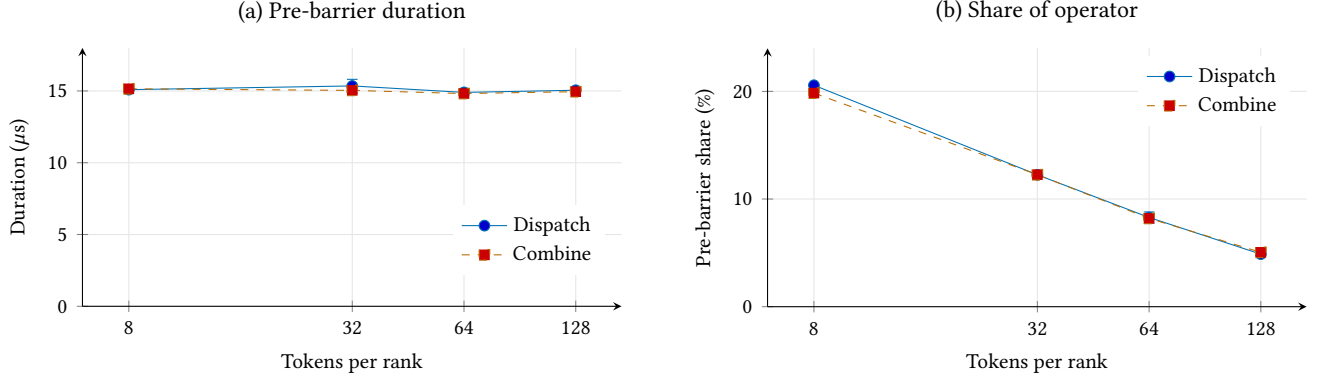


Figure 3. H200 cross-node EP8 Direct. Pre-barrier duration stays close to 15 μ s while its operator share decreases. Both use the selected rank’s trace-on measurements; error bars are standard deviations across three run means. Lines connect measured points.

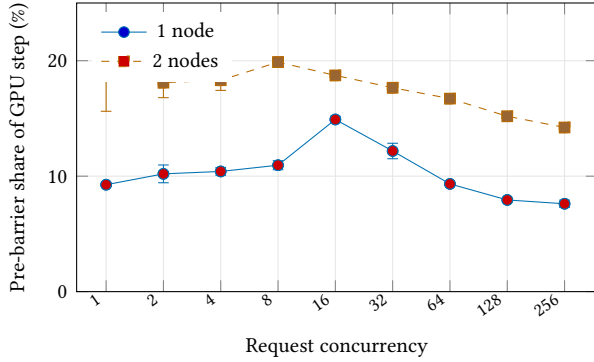


Figure 4. Qwen3-30B-A3B decode at fixed EP8. Each point sums the pre-barriers from 48 Dispatch and 48 Combine calls in one sampled GPU step. Both sweeps cover concurrency 1–256; low-concurrency windows may activate fewer than all ranks. Lines connect measured points. Error bars show variation across three client windows.

5.4 Pre-Barriers in MoE Serving

We measure Qwen3-30B-A3B in BF16 with vLLM 0.27 and DeepEP V2 Direct. The model has $H = 2048$, 128 experts, and $topk = 8$. Runs use actual routing, $TP=1$, EP8, and requests with 128 input and 128 output tokens. Figure 4 reports one-node and cross-node sweeps from concurrency 1 to 256.

The serving build includes tracing and an expert-padding correctness fix. For decode, one active GPU step is sampled per rank and client window; each sampled step contains 48 Dispatch and 48 Combine calls. We sum the pre-barrier intervals from those calls, divide by the same rank’s trace-on GPU-step time, and average over active rank-steps and three client windows on the same server instance. Thus Figure 4 and Table 2 measure cumulative pre-barrier exposure within an MoE GPU step, whereas Figure 3 measures a single communication operator.

EP	Workload	T/rank	Pre-barrier / step
4	Decode C64	16	$14.75 \pm 0.21\%$
8	Decode C128	16	$15.19 \pm 0.05\%$
16	Decode C256	16	$15.04 \pm 0.12\%$
8	Prefill	512	$2.64 \pm 0.06\%$
8	Prefill	2048	$3.13 \pm 0.39\%$

Table 2. Cross-node serving coverage. Decode rows match 16 tokens per rank across EP configurations. Prefill uses concurrency one and reports its active step. Values are means and standard deviations across three client windows.

For one node, the share stays between 7.61% and 14.91% across the sweep. Across two nodes, it stays between 14.22% and 19.88%. C1 is a low-load anchor with one active rank in each sampled window; from C16 onward both topologies activate all eight ranks. Readiness coordination therefore remains visible with real routing and model computation, and the cross-node path exposes a larger fraction of the GPU step.

Table 2 extends the cross-node results. At 16 tokens per rank, the share remains near 15% across EP4, EP8, and EP16. Prefill has much larger token batches and a 2.6–3.1% share. Each prefill row samples the single active prefill step per client window; it is a workload comparison rather than a controlled token-only experiment.

The serving percentages average over ranks, whereas the microbenchmark uses rank-minimum intervals. Together, they show a nearly fixed operator-level pre-barrier cost and its larger exposure in small-token decode.

6 Related Work

6.1 EP Communication

DeepEP [3] and NCCL EP [5] provide GPU-initiated communication paths for expert parallelism. Their metadata exchanges, preallocated buffers, and synchronization schedules determine when data can be sent. RendezEP uses network staging to begin upload while peer readiness is still being collected.

SwiftEP [10] optimizes buffer movement and GPU work, while COMET [18] overlaps communication with computation. Perseus [14] reorganizes PUT, fence, and signal processing in the transfer pipeline. UBEP [11] uses point-to-point signals and Data-as-Flag to reduce stage synchronization in scale-up EP. These approaches improve data movement or its coordination; early upload complements them when a readiness dependency precedes transmission.

6.2 In-Network Computation

SHARP [16] and SwitchML [15] use network computation for collective reduction. DySHARP [17] combines dynamic addressing, multicast, and reduction for MoE, while MultiWrite [2] uses multicast writes to reduce repeated upstream transfers. These systems demonstrate network processing and state management for distributed workloads. RendezEP applies temporary state and buffering to overlap pre-barrier coordination with upload.

6.3 Synchronization and Delay Analysis

EPIC [1] provides protocol and resource abstractions for Ethernet in-network computing. GPU-initiated networking [6] provides endpoint communication primitives. RendezEP combines these capabilities with readiness checks, symmetric receive buffers, and flow control.

Swift [9] separates processing and network contributions to delay. We apply that distinction to the execution order of an EP operation. FabricPerf [7] further motivates explicit timing boundaries; our pre-barrier measurements pair each interval with the corresponding operator or GPU step.

7 Conclusion

H200 measurements show that pre-barriers remain near 15 μ s for cross-node EP8 Direct over the measured small-batch range and account for 15.2–16.7% of sampled cross-node MoE decode time. RendezEP lets locally ready ranks upload while INC collects readiness signals, buffering early data until destination writes are safe. This changes the order of coordination and transmission without requiring receive-buffer addresses to be recomputed each round. The analysis relates the overlap to operation completion time and identifies staging capacity, readiness skew, and forwarding throughput as the main factors governing its benefit.

References

- [1] 2026. EPIC: Programmable Protocols and Resource Abstractions for Ethernet In-Network Computing. arXiv:2605.18683. <https://arxiv.org/abs/2605.18683>.
- [2] 2026. MultiWrite: Eliminating Redundant Upstream Traffic in Many-to-Many Communication. arXiv:2605.22428. <https://arxiv.org/abs/2605.22428>.
- [3] DeepSeek-AI. 2026. DeepEP: An Efficient Expert-Parallel Communication Library. <https://github.com/deepseek-ai/DeepEP>.
- [4] DeepSeek-AI. 2026. DeepEP V2 synchronization and kernel sources. Commit 01dc3aaac820. Files: common/comm.cuh, impls/hybrid_dispatch.cuh, impls/hybrid_combine.cuh. https://github.com/deepseek-ai/DeepEP/tree/01dc3aaac8206802035dce2c302e38153c0bfbaa/deep_ep/include/deep_ep.
- [5] Amos Goldman, Nimrod Boker, Maayan Sheraizin, et al. 2026. NCCL EP: Towards a Unified Expert Parallel Communication API for NCCL. arXiv:2603.13606. <https://arxiv.org/abs/2603.13606>.
- [6] Khaled Hamidouche, John Bachan, Pak Markthub, Peter-Jan Gootzen, Elena Agostini, Sylvain Jauegy, Aamir Shafi, Georgios Theodorakis, and Manjunath Gorenthla Venkata. 2025. GPU-Initiated Networking for NCCL. arXiv:2511.15076. <https://arxiv.org/abs/2511.15076>.
- [7] Songlin Huang and Chenshu Wu. 2026. FabricPerf: Measuring NIC-less Scale-Up Network through GPU Communication Kernel Profiling. Project repository. <https://github.com/open-neutrino/fabricperf>.
- [8] IEEE 802.1 Working Group. 2011. IEEE 802.1Qbb: Priority-based Flow Control. <https://1.ieee802.org/dcb/802-1qbb/>.
- [9] Gautam Kumar, Nandita Dukkipati, Keon Jang, Hassan M. G. Wasel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *ACM SIGCOMM*. 514–528. <https://doi.org/10.1145/3387514.3406591>.
- [10] Xingyi Li, Yadong Liu, Xiaojie Huang, et al. 2026. SwiftEP: Accelerating MoE Inference with Buffer Fusion and TMA Offloading. In *USENIX NSDI*. <https://www.usenix.org/conference/nsdi26/presentation/li-xingyi>.
- [11] Yipeng Liu, Chang Liu, Si Shen, et al. 2026. UBEP: Re-architecting Expert Parallelism Communication Library for Production Superpods. arXiv:2607.06202. <https://arxiv.org/abs/2607.06202>.
- [12] NVIDIA. 2026. NCCL EP HT device implementation. Commit 7b83616df3ae. https://github.com/NVIDIA/ncccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/contrib/ncccl_ep/device/hybrid_ep.cuh.
- [13] NVIDIA. 2026. NCCL EP LL device implementation. Commit 7b83616df3ae. https://github.com/NVIDIA/ncccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/contrib/ncccl_ep/device/ll_ep.cuh.
- [14] Byungsoo Oh and Rachee Singh. 2026. Eliminating Hidden Serialization in Multi-Node Megakernel Communication. arXiv:2605.00686. <https://arxiv.org/abs/2605.00686>.
- [15] Amedeo Sapia, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtarik. 2021. Scaling Distributed Machine Learning with In-Network Aggregation. In *USENIX NSDI*. 785–808. <https://www.usenix.org/conference/nsdi21/presentation/sapia>.
- [16] Scot Schultz. 2024. Advancing Performance with NVIDIA SHARP In-Network Computing. <https://developer.nvidia.com/blog/advancing-performance-with-nvidia-sharp-in-network-computing>.
- [17] Qijun Zhang, Chen Zhang, Zhuoshan Zhou, et al. 2026. Accelerating MoE with Dynamic In-Switch Computing on Multi-GPUs. arXiv:2605.05607. <https://arxiv.org/abs/2605.05607>.

- [18] Shulai Zhang, Ningxin Zheng, Haibin Lin, et al. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. arXiv:2502.19811. <https://arxiv.org/abs/2502.19811>.